Briefing: Cyclistic Case Study — SQL Server & SSIS Module

For: Claude Employment/Portfolio Project

From: Cyclistic Case Study Project

What This Is

Peter is building a multi-tool data analytics portfolio capstone based on the Google Data Analytics Certificate case study (Cyclistic bike-share, based on real Divvy/Lyft data from Chicago). The project analyzes 12 months of ride data (April 2025 – March 2026, 5.6M+ records) to answer the business question: *how do annual members and casual riders use Cyclistic bikes differently, and how can the company convert casual riders to members?*

The project is being built in multiple tool phases to demonstrate breadth across the modern data analytics stack:

- ✅ **Power BI** — interactive report with DAX measures (complete)
- ✅ **SQL Server + SSIS** — enterprise ETL pipeline and T-SQL analysis (just completed)
- 🔜 **DuckDB** — ad-hoc CSV analysis without a database server
- 🔜 **R** — statistical analysis and visualization
- 🔜 **Python** — scripted analysis and automation

This briefing covers what was accomplished in the SQL Server + SSIS phase.

What Was Built

SSIS ETL Pipeline (`LoadTrips.dtsx`)

A production-grade ETL package built in Visual Studio / SSDT that:

- Iterates 12 monthly CSV source files via a Foreach Loop File Enumerator
- Loads raw data into a staging table (`stg.Trips`) — all NVARCHAR, no constraints
- Validates staging row count before proceeding (aborts if below threshold)
- Transforms and loads into a production table (`dbo.Trips`) with full type casting, NULL filtering, and two computed columns
- Redirects Data Flow error rows to `stg.Trips_Errors`
- Logs package-level failures to `dbo.ETL_ErrorLog` via an OnError event handler

This is a staging-to-production pattern that mirrors enterprise data warehouse practice.

SQL Server Database Schema

Five tables across two schemas (`stg` and `dbo`):

- Surrogate integer primary key with IDENTITY
- Natural key UNIQUE constraint on `ride_id`
- CHECK constraint on `member_casual`
- Non-clustered indexes on the four most-queried columns
- Two computed columns calculated during ETL load:
  - `ride_time_min` — decimal minutes via DATEDIFF, NULL for negative durations
  - `ride_distance_mi` — Haversine formula in miles, NULL for missing coordinates

Python Query Runner (`query_runner.py`)

A utility script connecting to SQL Server via `pymssql` that runs `.sql` files and saves results as timestamped CSVs. Includes GO-separator splitting (pymssql doesn't natively handle T-SQL batch separators). Used for all query development and validation.

14 Analytical T-SQL Queries

Written to replicate and validate the Power BI DAX measures, covering:

- Core ride counts and member/casual split
- Average duration and distance by rider type
- Weekend vs. weekday distribution
- Round trip analysis
- Hourly ride distribution
- Seasonal breakdown (monthly + season rollup)
- Bike type mix
- Top 10 start stations by member and casual separately
- Missing station data analysis
- Revenue proxy estimation

Queries use CTEs throughout — including a CROSS JOIN pattern for clean percentage calculations that avoids nested window aggregates.

Documentation

- `etl_process.md` — full ETL documentation including architecture decisions, step-by-step control flow, transformation logic (Haversine formula shown), error handling, and notable challenges resolved
- `cross_validation.md` — systematic validation of every SQL result against the Power BI report with match/discrepancy table
- `schema.md` — full database schema reference
- `README.md` — module overview with cross-validation findings highlighted

---

Key Technical Skills Demonstrated

- **SSIS** — Foreach Loop, Data Flow (Flat File Source, Derived Column, OLE DB Destination), Execute SQL Task, row count validation, error output redirection, OnError event handler, package variables, connection managers
- **SQL Server** — schema design, IDENTITY PKs, UNIQUE and CHECK constraints, non-clustered indexes, CTEs, window functions, CROSS JOIN for aggregation, TRY_CAST, Haversine formula in T-SQL, ISNULL,

NULLIF, DATEPART
- **Python** — pymssql, python-dotenv, batch splitting for T-SQL GO separators, CSV output
- **Git/GitHub** — source control for SQL scripts, SSIS package, and Python utilities

---

Analytical Findings Worth Highlighting on Resume/Portfolio

These are concrete, data-backed findings — not generic observations:

Behavioral differences between members and casuals:

- Casual riders take **82% longer rides** on average (22.6 min vs 12.4 min)
- Members show a **dual commute peak** (8 AM and 5 PM); casuals show a **single leisure peak** (3–5 PM)
- Casuals are **61% more likely to ride on weekends** than members
- Casuals take **round trips at 1.63× the rate** of members
- Casual ridership drops to just **19.6% of winter rides** vs 42.7% in summer — far more weather-sensitive than members

Geographic findings:

- **Shedd Aquarium station: 81.8% casual** — the single starkest example. Every station in the casual top 10 is a lakefront tourist destination (Navy Pier, Millennium Park, DuSable Lake Shore Drive, etc.)
- **Zero overlap** between the member top 10 and casual top 10 station lists
- Member top stations cluster around **Union Station and Ogilvie Transportation Center** — Chicago's commuter rail hubs
- Clinton St & Washington Blvd: **83.3% member** — the mirror image of Shedd Aquarium

Data quality / technical findings:

- **Inferred e-bike GPS hardware from data structure**: electric bikes have ~32.5% missing station names but fully populated lat/lng coordinates. Classic bikes have 0% missing station data. The only explanation is that e-bikes have onboard GPS — a real analytical inference drawn from the data, not assumed from external knowledge.

The Most Portfolio-Worthy Discovery

During cross-validation of the round trip metric, the initial SQL query produced results dramatically different from Power BI (2% vs 11% for members). Rather than accepting the discrepancy, a hypothesis was formed: **Power BI DAX treats BLANK() = BLANK() as TRUE**, while SQL Server's three-valued logic treats NULL = NULL as UNKNOWN.

A targeted diagnostic query (`08b_round_trips_diag.sql`) tested three definitions side by side and confirmed the hypothesis exactly. The production query was updated to use `ISNULL()` to replicate DAX behavior, producing an exact match (11.1% / 18.1% / ratio 1.63).

This demonstrates the kind of cross-tool analytical rigor that separates someone who runs queries from someone who investigates results — forming a hypothesis, building a test, interpreting the outcome, and correcting the implementation. It also produced a genuine insight: rides where both stations are blank aren't data gaps, they're e-bikes returned to non-station racks, and they legitimately count as round trips.

How to Use This for Resume/Portfolio/Upwork

Resume bullet suggestions:

- Built enterprise ETL pipeline in SSIS processing 5.6M records across 12 monthly CSV files using staging-to-production pattern with full error handling and audit logging
- Wrote 14 T-SQL analytical queries using CTEs, window functions, and Haversine formula to validate Power BI DAX measures against SQL Server results
- Diagnosed and resolved cross-tool metric discrepancy by identifying semantic difference between DAX blank equality and SQL three-valued null logic; confirmed with targeted diagnostic query
- Inferred e-bike GPS hardware capabilities from data structure (coordinates present without station names) demonstrating ability to reason about data collection mechanisms

Portfolio narrative angle: The SQL Server module is strongest as a demonstration of *rigor* — not just completing the analysis, but verifying it independently across tools, investigating discrepancies, and documenting everything to production standards. The cross_validation.md document is the artifact that makes this concrete.

Upwork angle: Relevant for clients who need SQL Server / SSIS work, ETL pipeline development, data validation, or Power BI back-end work. The combination of SSIS + T-SQL + Python query runner + documentation shows someone who can own a full data pipeline from ingestion to analysis.

Project Location

`D:\OneDrive\Job Stuff\Portfolio Stuff 2026\Google Cert Capstone Case Study 1 - Cyclist\`

Key subfolders:

- `SQLServer\` — all SQL Server module files
- `SQLServer\SSIS\CyclisticETL\LoadTrips.dtsx` — the SSIS package
- `SQLServer\queries\` — all 14+ T-SQL queries
- `SQLServer\schema\` — DDL scripts
- `SQLServer\etl_process.md` — ETL documentation
- `SQLServer\cross_validation.md` — Power BI validation results